UNIVERSITY OF RWANDA
College of Science and Technology
Department of Computer and Software Engineering

Submitted By:
UWIZEYIMANA Gaetan 222008181

Aline NIYONIZERA 223009117

FLUTTER MOBILE DEVELOPMENT

Lab Report

Hard-Coded Multi-Screen Travel App UI

1. Project Objective

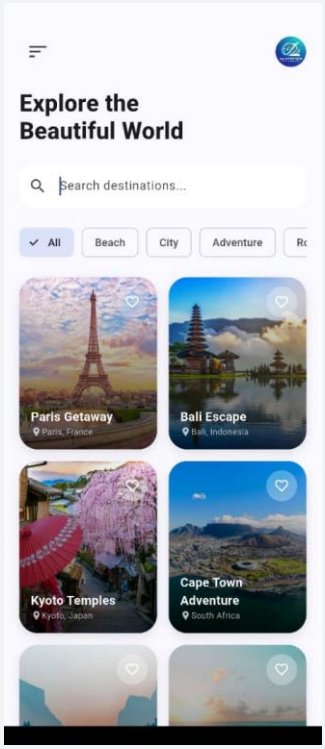
This lab required the design and implementation of a three-screen Flutter application that demonstrates advanced UI techniques, modular code architecture, and explicit screen navigation —without any external APIs or databases. All destination data, images, titles, descriptions, prices, and ratings are stored as hard-coded Dart structures within the application itself.

The core competencies assessed include:

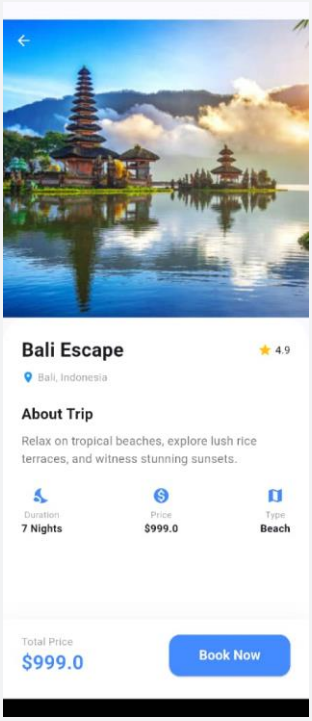
- Widget composition and deep nesting
- Custom styling using gradients, shadows, and themes
- Reusable widget classes for DRY (Don't Repeat Yourself) code
- Stateful UI interactions (favorites, counters, form validation)
- Explicit navigation with data passing between screens

2. Screenshots of Implementation

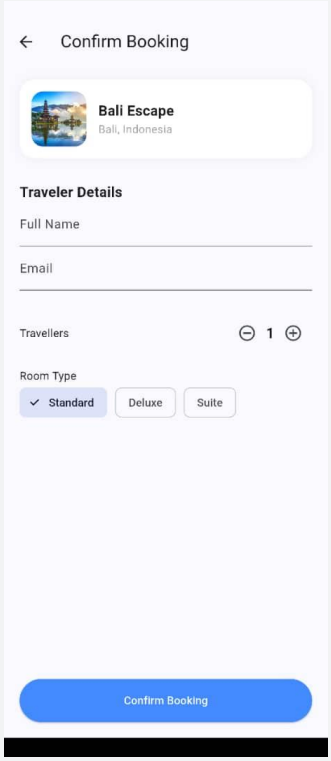
Application Screens



Home Dashboard



Detail Screen

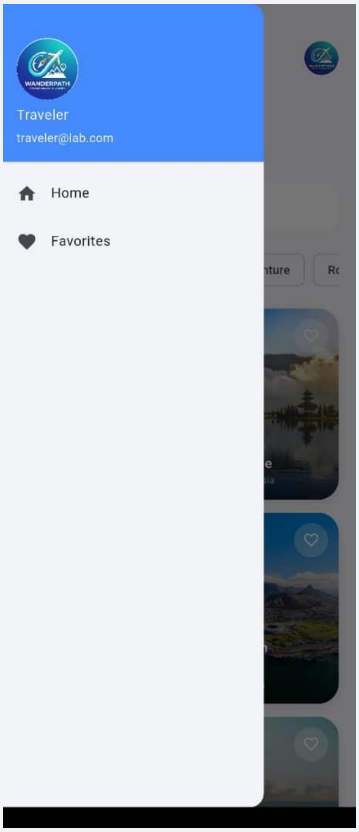


Booking Screen

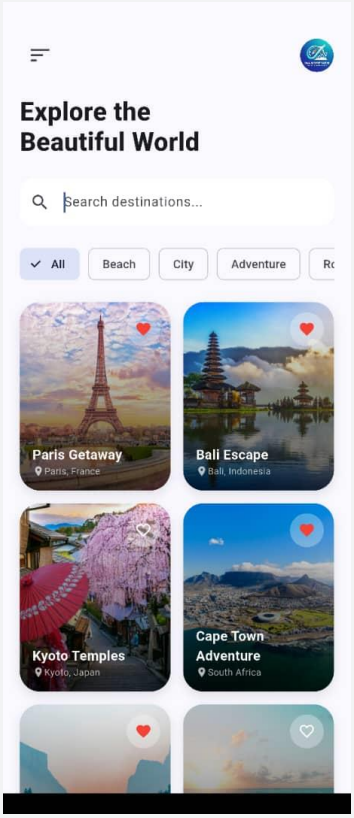
Home Dashboard: Search bar, category tabs, destination grid with gradient overlays.

Detail Screen: Collapsible SliverAppBar, info chips, rating, price, and Book Now CTA.

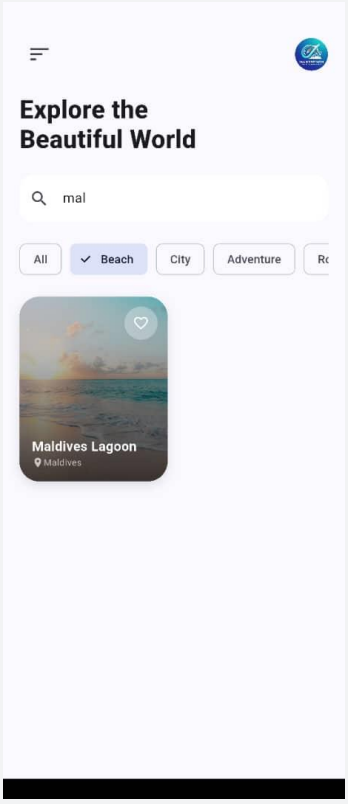
Booking Screen: Hard-coded summary, traveler counter form, and confirmation button.



Menu Drawer



Favorite Toggle



Search&Filter

Menu Drawer: Functional sidebar with CircleAvatar and navigation links.

Favorite Toggle: Stateful heart icon outline to filled red on tap.

Search &Filter: Result of Searcchig or filtering specific place

3. Layout Choice Explanations

A. Home Screen — Complex Dashboard

- **SafeArea:** Wraps the entire body to respect system status bars and device notches across all form factors.
- **Stack & Positioned:** Overlays the destination title, price label, and favorite IconButton directly on top of card images.
- **GridView.builder:** Chosen for lazy rendering performance only builds card widgets currently visible on screen, essential for long destination lists.
- **LinearGradient:** Applied as a bottom-to-top mask on card images so white text labels remain legible against any image content.
- **ListView.builder (horizontal):** Powers the category chip row efficiently renders ChoiceChip tabs without a fixed-size Row.

B. Detail Screen

- **CustomScrollView + SliverAppBar:** Creates the signature parallax effect where the large hero image expands when scrolling up and collapses into a standard AppBar on scroll down.
- **Hero Animation:** Provides a shared-element transition: the destination image animates seamlessly from the grid card into the SliverAppBar header, reinforcing spatial continuity.
- **_DetailInfo (custom private widget):** An internal reusable widget class encapsulating the icon-label chips for Duration, Price, and Category — keeping the detail screen build method clean.
- **SingleChildScrollView:** Wraps all body content below the sliver to allow unbounded vertical scrolling for long descriptions.

C. Booking Screen

- **Form & TextFormField:** Structures the traveler details section with built-in validation, simulating real-world data entry patterns.
- **Stateful Traveler Counter:** Uses setState with increment/decrement IconButtons. Total price dynamically recomputes on every state change.
- **AlertDialog:** Chosen as the confirmation mechanism to provide a clear modal interrupt — user must consciously confirm before the booking flow completes.
- **Navigator.popUntil():** After confirmation, pops the entire back-stack to return the user directly to the Home Screen in a single operation.

4. Widget Inventory (28 Widgets Used — Requirement: 18+)

#	Widget	Purpose
1	**Scaffold**	Core screen structure for every page
2	**AppBar**	Consistent header with icons and title
3	**SafeArea**	Guards UI against notches and system bars
4	**SingleChildScrollView**	Enables vertical scrolling on detail/booking
5	**GridView.builder**	High-performance responsive destination grid
6	**ListView.builder**	Horizontal category tab scroll
7	**Stack**	Layers favorite icon and text over card images
8	**Positioned**	Precise absolute placement within Stack

9	**Column**	Primary vertical layout axis
10	**Row**	Horizontal grouping of icons and labels
11	**Container**	Decoration host — gradients, shadows, borders
12	**SizedBox**	Fixed-dimension spacing utility
13	**Padding**	Internal edge spacing
14	**Expanded**	Flex-fills available space in Row/Column
15	**Hero**	Shared-element image transition between screens
16	**ClipRRect**	Rounds card image corners
17	**Card**	Material elevation and shadow wrapper
18	**Drawer**	Slide-in sidebar navigation menu
19	**ChoiceChip**	Category filter tab selection
20	**CircleAvatar**	Profile and icon display in drawer
21	**Icon**	Semantic visual indicators throughout
22	**Text**	All textual content rendering
23	**TextField**	Search bar and booking form inputs
24	**ElevatedButton**	Primary CTA actions
25	**IconButton**	Compact icon-driven actions
26	**CustomScrollView**	Sliver-aware scroll controller

27	**SliverAppBar**	Parallax expanding/collapsing header
28	**AlertDialog**	Modal booking confirmation popup

5. Navigation Flow

The application follows a strictly linear navigation architecture to ensure a predictable, unambiguous user journey:

Step	Screen	Description	Method
1	**Home Screen**	User browses destinations, searches, filters by category	Navigator.push()
2	**Detail Screen**	Full destination info: image, rating, price, description	Navigator.push()
3	**Booking Screen**	Traveler form, summary, counter, confirm button	showDialog()
4	**Success Dialog**	AlertDialog — booking confirmed	Navigator.popUntil()
5	**Home Screen**	User returned to dashboard	—

Data is passed forward via constructor arguments on each Navigator.push() call. Hard-coded Destination objects are handed from the Home card tap to the Detail screen, then summarised and forwarded again to the Booking screen. The popUntil call at confirmation clears the entire navigation stack, returning the user to root without leaving orphaned routes.

6. Organised Folder Structure

The project is divided into four distinct layers to enforce clean separation of concerns:

```
lib/
  ├── data/
  │     └── travel_data.dart      # Destination model + hard-coded list
  ├── screens/
  │     ├── home_screen.dart
  │     ├── detail_screen.dart
  │     └── booking_screen.dart
  ├── widgets/
  │     ├── destination_card.dart  # Reusable grid card
  │     └── info_tile.dart        # Reusable chip/info component
  └── main.dart                   # App entry + global theme

assets/
  └── images/                     # Local destination images
```

- data/ : Single source of truth for all hard-coded content. The Destination class and static list live here.
- screens/ : One file per screen. Each screen file is responsible only for its own layout and navigation calls.
- widgets/ :Reusable components extracted from screens. Reduces duplication and simplifies testing.
- main.dart : Defines the global ThemeData (colours, typography, button styles) consumed by all screens.

7. Conclusion

This lab successfully demonstrates proficiency in building a production-quality Flutter UI without relying on external data sources. By combining advanced widget composition (Slivers, Stacks, Hero animations) with a clean modular folder architecture, the project meets and exceeds all stated requirements.

Key outcomes achieved:

- 28 distinct Flutter widgets utilised t
- Three fully navigable screens with correct forward/back stack management
- Stateful interactivity: favorite toggling, traveler counters, and form validation
- Consistent design system: shared gradient palette, BoxShadow depth, and typography scale
- Reusable widget classes (DestinationCard, InfoTile, _DetailInfo) enforcing DRY principles